

Azure Databricks

Lakehouse architecture, Delta Lake, Unity Catalog, engineering, streaming, and operations

Curated study notes for the RAG learning project. Synthesized from official documentation; verify current product behavior in official docs before production use.

1. Azure Databricks Overview

Azure Databricks is a managed analytics platform built around Apache Spark and integrated with Azure services. It provides collaborative development, data engineering, analytics, machine learning, governance, and production orchestration.

Key idea: Think of Azure Databricks as a managed platform around Spark, storage, governance, notebooks, jobs, and data/AI workflows—not simply a hosted notebook.

Core ideas

A workspace is the collaborative environment where users work with notebooks, compute, jobs, and other assets. The account layer manages broader organizational configuration and governance.

- Workspaces provide the user-facing environment.
- Compute executes workloads.
- Unity Catalog provides centralized governance for data and AI assets.
- Jobs/Lakeflow orchestration automate recurring workloads.

Lakehouse model

The lakehouse combines the low-cost, flexible storage of a data lake with reliability, governance, and analytical performance associated with warehouses. Delta Lake is central to the Databricks implementation.

- Raw data can be retained in object storage.
- Curated Delta tables can provide reliable analytical data.
- SQL, notebooks, pipelines, and ML workloads can share governed data.

Azure integration

Azure Databricks commonly integrates with Azure Data Lake Storage, Microsoft Entra ID, Key Vault, networking, Data Factory, Power BI, and other Azure services.

- Separate compute from storage.
- Use managed identity/service principals and governed access where possible.
- Design networking and data access according to enterprise security requirements.

2. Azure Databricks Architecture

Azure Databricks separates the control plane from the compute plane and organizes data governance through Unity Catalog. This separation helps explain where metadata, orchestration, and execution occur.

Key idea: A workspace is not the same thing as a cluster, and Unity Catalog is not simply a folder. They solve different architectural problems.

Control and compute planes

The control plane contains services managed by Databricks for the account/workspace experience. The compute plane runs customer workloads and processes data.

- Users interact with workspace services.
- Compute resources execute notebooks, jobs, and queries.
- Network design affects how compute reaches storage and other services.

Unity Catalog hierarchy

Unity Catalog organizes governed objects using a three-level namespace: catalog.schema.object. It provides a central model for permissions and discovery across workspaces connected to a metastore.

- Catalogs provide a top-level organizational boundary.
- Schemas group related tables, views, volumes, and other objects.
- Tables and views are the common analytical objects.
- Privileges should follow least-privilege principles.

Metadata vs data

The catalog stores metadata and governance information; the actual table data remains in storage. Understanding this distinction prevents confusion about where data physically lives.

- Managed and external tables have different ownership/storage patterns.
- External locations connect governed access to cloud storage paths.
- Use governed access rather than hard-coding storage credentials in notebooks.

3. Compute, Clusters, Warehouses, and Serverless

Databricks provides compute choices for different workloads. Spark clusters are suited to engineering and distributed processing; SQL warehouses provide SQL-focused compute; serverless options reduce infrastructure management where available.

Key idea: Choose compute based on workload, isolation, latency, cost, and governance requirements—not simply on the largest cluster available.

Classic Spark compute

Spark compute runs distributed workloads such as ETL, batch processing, streaming, and notebook experiments.

- Driver coordinates the Spark application.
- Worker/executor resources perform distributed tasks.
- Autoscaling can adjust resources for changing workload demand where supported.

SQL warehouses

SQL warehouses provide managed compute optimized for SQL analytics and BI-style workloads.

- Separate SQL compute can serve analysts without sharing a job cluster.
- Warehouse sizing and auto-stop settings influence cost and latency.
- Use query history and monitoring to tune workload behavior.

Serverless

Serverless compute abstracts infrastructure management and can reduce operational overhead. Exact availability and feature support vary by workload and region.

- Verify current feature support before designing around a serverless-only capability.
- Serverless does not remove the need for data governance and query optimization.

Cost discipline

Idle compute can become a major cost driver. Cluster policies, auto-termination, right-sizing, and workload-specific compute are important operational controls.

- Use job-specific compute for scheduled workloads where appropriate.
- Avoid using expensive interactive clusters for unattended batch jobs.
- Monitor DBUs/cloud infrastructure cost and workload duration.

4. Notebooks, Repos, Jobs, and Orchestration

Databricks supports interactive notebooks for exploration and development and production orchestration for repeatable workloads.

Key idea: A notebook is a development interface; a production data pipeline needs versioning, parameters, scheduling, monitoring, retries, and controlled deployment.

Notebooks

Notebooks combine executable code, documentation, visualizations, and results. They are excellent for exploration and collaborative development.

- Use parameters for reusable logic.
- Avoid hidden state between cells.
- Keep production logic modular and testable.

Version control

Git integration and deployment tooling can connect notebooks and source code to a software-development lifecycle.

- Review changes.
- Use branches and pull requests where organizational practice requires them.
- Separate development, test, and production environments.

Jobs/Lakeflow Jobs

Jobs orchestrate tasks with schedules, triggers, dependencies, parameters, retries, and monitoring. Modern Databricks terminology and capabilities evolve, so consult current documentation for exact UI/API names.

- Make task dependencies explicit.
- Use retries for transient failures, not for deterministic data-quality failures.
- Capture run metadata and output locations.

Production pattern

A typical workflow is source ingestion → bronze/raw table → validation/cleaning → silver/curated table → business aggregates → serving/BI.

- Keep transformations deterministic where possible.
- Use checkpoints for streaming.
- Design reruns and backfills deliberately.

5. Delta Lake Fundamentals

Delta Lake is a storage layer that adds transactional reliability and table-management capabilities to data stored in a data lake. Databricks uses Delta extensively for lakehouse tables.

Key idea: Delta tables are still files in cloud storage, but Delta's transaction log enables a table abstraction with atomic commits, schema management, and time-travel-style capabilities.

Transaction log

Delta maintains a transaction log that records table changes. Readers use the log to determine a consistent table snapshot.

- Atomic commits reduce partial-write problems.
- Concurrent operations can be coordinated through transaction semantics.
- The log also enables table history and version-aware operations.

ACID-style behavior

Delta provides atomicity, consistency, isolation, and durability semantics for supported operations. This is particularly useful when multiple jobs interact with the same analytical tables.

- Reliable writes are important for pipelines.
- Readers can see consistent snapshots.
- Operational practices still matter: bad transformations can produce logically incorrect data even when writes are atomic.

Schema enforcement/evolution

Delta can enforce schema expectations and can support controlled schema evolution. Schema changes should be governed rather than allowed blindly.

- Schema enforcement catches incompatible writes.
- Evolution is useful when upstream contracts legitimately change.
- Document breaking changes and downstream impact.

Time travel and maintenance

Table history can support reproducibility and debugging. Maintenance operations such as compaction and cleanup must be planned with retention and recovery requirements in mind.

- Do not treat cleanup as an arbitrary delete of old files.
- Understand retention implications before removing historical data.
- Use official guidance for production retention and recovery settings.

6. Unity Catalog and Data Governance

Unity Catalog provides centralized governance and discovery for data and AI assets across Databricks workspaces. It is a foundational part of enterprise Databricks architecture.

Key idea: Governance is about who can discover, read, write, share, and administer data—not merely where the files are stored.

Three-level namespace

The catalog.schema.object hierarchy creates a consistent naming model for governed assets.

- Example: analytics.sales.orders.
- Use catalogs for organizational boundaries such as business domains or environments where appropriate.
- Schemas group related objects and permissions.

Privileges

Access can be granted through privileges on catalogs, schemas, tables, views, volumes, and other governed objects. Group-based access is generally easier to maintain than individual grants.

- Follow least privilege.
- Separate read, write, and administration rights.
- Review inherited and object-level privileges carefully.

Storage governance

External locations and storage credentials allow governed access to cloud storage paths. This reduces the need to expose raw storage credentials in notebooks.

- Avoid secrets embedded in source code.
- Use managed identities/service principals when supported by the architecture.
- Separate production and non-production storage access.

Governance outcomes

A mature setup provides discoverability, ownership, auditability, lineage, access control, and consistent policies.

- Governance should be designed alongside pipelines, not added after deployment.
- Metadata quality matters: owners, descriptions, classifications, and freshness improve usability.

7. Ingestion and Data Engineering in Databricks

Databricks can ingest batch and streaming data from cloud storage, databases, message systems, and managed connectors. The engineering goal is to create reliable, observable, repeatable data flows.

Key idea: Ingestion is not just copying bytes. It includes schema handling, deduplication, checkpoints, error handling, metadata, and operational guarantees.

Batch ingestion

Batch jobs commonly read files or source tables, apply transformations, and write Delta tables.

- Use incremental patterns when full reloads are unnecessary.
- Track source watermarks or modification timestamps.
- Validate row counts and key constraints where practical.

Streaming ingestion

Structured Streaming can incrementally process new data. Checkpoints record progress and state so the query can recover.

- Choose a checkpoint location that is durable and isolated per query.
- Do not casually reuse checkpoints across incompatible query logic.
- Monitor input rate, processing rate, latency, and state size.

Bronze/silver/gold

The medallion pattern is a common organization: bronze preserves relatively raw data, silver applies cleansing/conformance, and gold provides business-ready aggregates or marts.

- Bronze improves traceability.
- Silver is where data quality and standardization are emphasized.
- Gold should serve clear analytical or operational use cases.

Data quality

Checks can cover nulls, uniqueness, referential relationships, valid ranges, schema compatibility, freshness, and duplicate detection.

- Record failed records rather than silently dropping them.
- Make quality thresholds visible to operators.
- Treat data-quality rules as code and version them.

8. Transformations, SQL, and Lakehouse Modeling

Databricks supports DataFrame APIs, SQL, and declarative pipeline patterns for transforming data. Good modeling reduces downstream complexity and improves performance.

Key idea: Model around access patterns and business meaning, not around the shape of one upstream file.

Common transformations

Filtering, projection, joins, aggregations, windows, deduplication, parsing, and enrichment are common Spark operations.

- Prefer built-in Spark functions over Python UDFs when an equivalent exists.
- Keep transformations composable and testable.
- Avoid unnecessary wide operations.

Dimensional modeling

Analytical models may use facts and dimensions, star schemas, or denormalized tables depending on workload.

- Facts represent measurable events.
- Dimensions describe entities and business context.
- Surrogate keys, slowly changing dimensions, and grain should be defined explicitly when required.

Incremental design

Incremental pipelines process only new or changed data when the source supports a reliable change signal.

- Define the incremental key or watermark.
- Handle late-arriving data.
- Make the write idempotent so reruns do not duplicate records.

SQL performance

Performance depends on file layout, statistics, partitioning, clustering/data skipping mechanisms, join strategies, and query shape.

- Select only required columns.
- Filter early.
- Avoid exploding data unnecessarily.
- Use query profiles and execution details rather than guessing.

9. Streaming and Stateful Processing

Databricks uses Structured Streaming for streaming workloads and supports event-driven data engineering patterns. Streaming design requires explicit thinking about delivery semantics, state, late data, and recovery.

Key idea: A streaming pipeline should answer four questions: what is the source, what state must be remembered, how is progress checkpointed, and what does the sink guarantee?

Micro-batch mental model

Many Structured Streaming workloads process data in small batches. Each trigger identifies new input and updates the result.

- Trigger intervals affect latency and cost.
- Backlog can grow when input arrives faster than processing capacity.
- Monitor both throughput and end-to-end latency.

Watermarks

Watermarks bound how long the engine keeps state for late events in supported operations.

- A larger watermark tolerates later data but retains more state.
- A smaller watermark reduces state but may drop late results depending on the query semantics.
- Choose based on business lateness expectations.

Exactly-once language

'Exactly once' is not a universal property of an entire pipeline. It depends on source guarantees, checkpointing, state, and sink commit semantics.

- Make sinks idempotent when practical.
- Use deterministic keys for deduplication.
- Document what happens during retries and partial failures.

Operations

Monitor stream progress, state, input rows, processing rates, failures, and checkpoint health.

- Alert on sustained lag.
- Plan checkpoint storage and permissions.
- Backfill historical data separately when that is safer than replaying a production stream.

10. Optimization and Cost Control

Databricks optimization combines Spark fundamentals with lakehouse storage and workload-specific features. The objective is lower runtime and cost without sacrificing correctness.

Key idea: Optimization is an evidence-driven loop: observe → identify bottleneck → change one thing → measure again.

File and table layout

Too many small files increase overhead. Poor locality can increase the amount of data scanned. Compaction and appropriate layout strategies can improve reads.

- Choose partitioning based on common filters and cardinality.
- Avoid high-cardinality directory partitions.
- Use current Databricks guidance for predictive optimization, liquid clustering, and related features where available.

Query optimization

Use explain plans, query profiles, Spark UI, and runtime metrics to find expensive scans, joins, shuffles, and spills.

- Broadcast small dimensions when appropriate.
- Reduce columns and rows early.
- Avoid unnecessary Python UDFs.
- Use AQE and current runtime optimizations where supported.

Cost controls

Compute cost is influenced by cluster size, runtime, idle time, workload concurrency, and storage/I/O behavior.

- Auto-termination reduces idle interactive cost.
- Use policies to enforce safe defaults.
- Separate interactive experimentation from scheduled production workloads.

Reliability before speed

A fast pipeline that produces duplicates or silently loses data is not optimized. Correctness, observability, and recoverability are production requirements.

- Benchmark representative data.
- Test backfills and reruns.
- Track SLA/SLO targets alongside cost.

11. Security, Identity, and Production Operations

Enterprise Databricks deployments combine cloud identity, workspace controls, network security, storage permissions, Unity Catalog governance, and operational monitoring.

Key idea: Use identity-based access and least privilege wherever possible; avoid putting long-lived credentials in notebooks or source repositories.

Identity

Azure environments commonly integrate with Microsoft Entra ID and service principals/managed identities. Exact supported patterns depend on the resource and deployment model.

- Prefer workload identities over personal credentials for automation.
- Rotate credentials when credentials are unavoidable.
- Use groups to manage access consistently.

Secrets

Secrets should be stored in an approved secret-management mechanism and referenced securely by workloads.

- Never commit API keys to Git.
- Do not print secrets in logs.
- Separate secret scopes/permissions by environment and application.

Network and storage

Private networking, firewall controls, managed endpoints, and storage permissions can restrict data access. The correct design depends on enterprise topology.

- Minimize public exposure.
- Use private connectivity where required.
- Ensure compute can access only the storage it needs.

Monitoring

Production operations require job monitoring, audit logs, query monitoring, cost visibility, and alerting.

- Record pipeline run IDs and input/output versions.
- Alert on repeated failures and SLA breaches.
- Keep operational dashboards understandable to on-call users.

12. Databricks End-to-End Reference Architecture

A practical Azure Databricks data-engineering architecture connects cloud storage, governed tables, Spark transformations, orchestration, and serving layers.

Key idea: The architecture should make lineage and recovery understandable: source → raw → curated → serving, with governance and observability across every layer.

Reference flow

Sources may include SaaS systems, databases, files, APIs, and event streams. Ingestion lands data in cloud storage. Databricks processes it into Delta tables governed by Unity Catalog.

- Source systems → ingestion → bronze.
- Bronze → quality/conformance → silver.
- Silver → business transformations → gold.
- Gold → SQL/BI/ML/serving consumers.

Orchestration

Jobs or external orchestrators can coordinate dependencies, schedules, retries, and parameterization. Azure Data Factory can also integrate with Databricks workloads.

- Keep orchestration logic separate from transformation logic.
- Use parameters for environment, date, and source selection.
- Make each task independently observable.

Governance

Unity Catalog provides names, permissions, lineage, and discovery around governed objects. Cloud storage remains the physical persistence layer.

- Catalogs/schemas provide logical organization.
- Access policies should reflect data sensitivity and business roles.
- Data owners should be identifiable.

Portfolio takeaway

For a data-engineering portfolio project, demonstrate ingestion, transformations, Delta tables, governance concepts, orchestration, tests, and monitoring rather than only a notebook with a few transformations.

- Document assumptions.
- Show an architecture diagram.
- Include failure/recovery cases.
- Measure runtime and data quality.

Selected official sources

- <https://learn.microsoft.com/en-us/azure/databricks/getting-started/high-level-architecture>
- <https://learn.microsoft.com/en-us/training/paths/azure-databricks-data-engineer-prepare-process-data/>
- <https://learn.microsoft.com/en-us/training/paths/azure-databricks-data-engineer-deploy-maintain-data-pipelines-workloads/>
- <https://docs.databricks.com/en/delta/>
- <https://docs.databricks.com/en/data-governance/unity-catalog/>